

✓ Data Visualization II

- i) Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data.
- ii) Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.
- iii) Use the inbuilt dataset 'titanic' as used in the above problem. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (column names: 'sex' and 'age')
- iv) Write observations on the inference from the above statistics.

✓ Downloading the Seaborn Library

The seaborn library can be downloaded in a couple of ways. If you are using pip installer for Python libraries, you can execute the following command to download the library:

```
pip install seaborn
```

Alternatively, if you are using the Anaconda distribution of Python, you can use execute the following command to download the seaborn library:

```
conda install seaborn
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
dataset = sns.load_dataset('titanic')
dataset.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

The dataset contains 891 rows and 15 columns and contains information about the passengers who boarded the unfortunate Titanic ship. The original task is to predict whether or not the passenger survived depending upon different features such as their age, ticket, cabin they boarded, the class of the ticket, etc. We will use the Seaborn library to see if we can find any patterns in the data.

```
dataset.shape
```

```
(891, 15)
```

```
dataset.isnull()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False
887	False	False	False	False	False	False	False	False	False
888	False	False	False	True	False	False	False	False	False
889	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False

891 rows × 15 columns

```
dataset.isnull().sum()
```

	0
survived	0
pclass	0
sex	0
age	177
sibsp	0
parch	0
fare	0
embarked	2
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

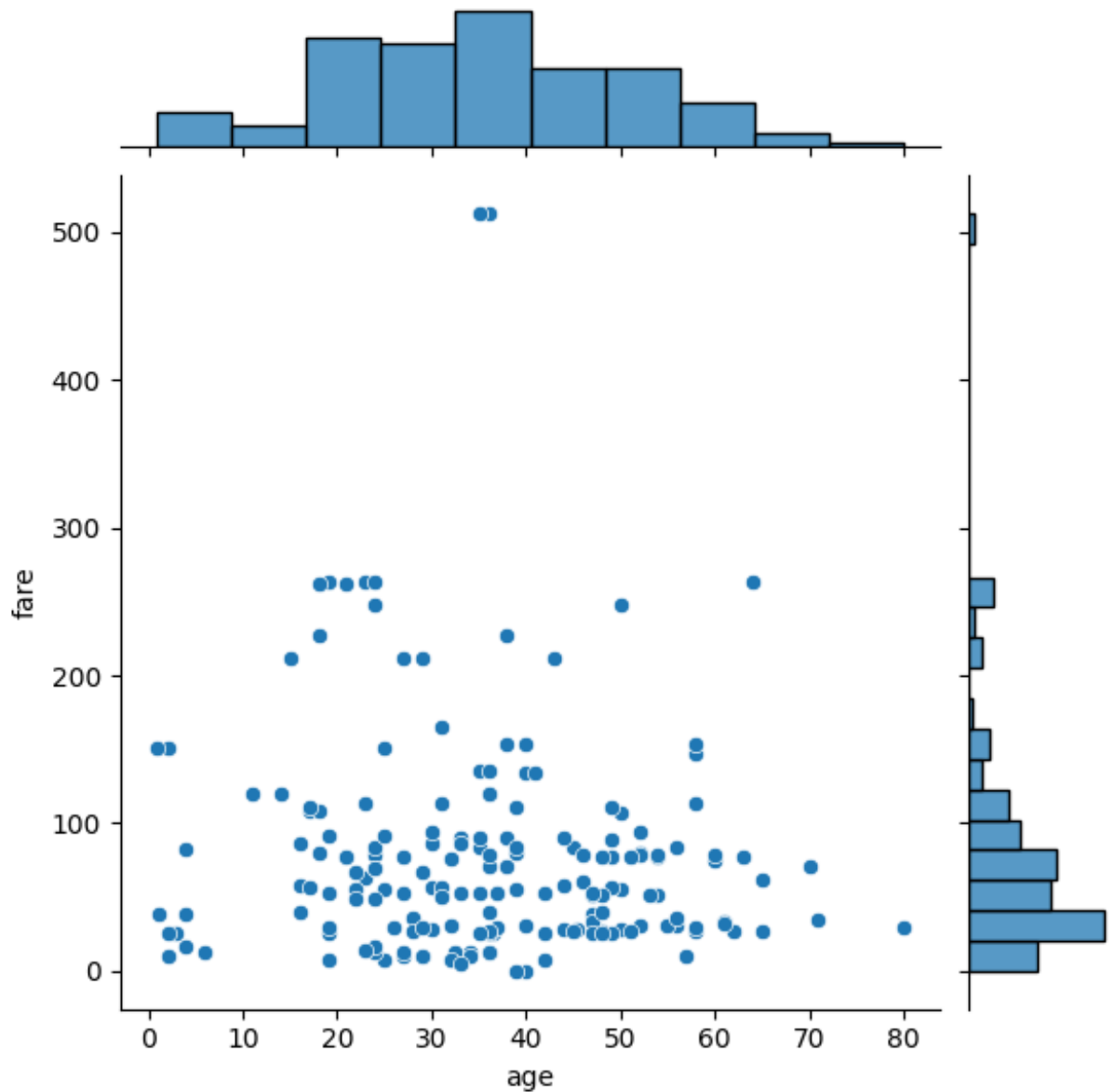
```
# remove all null values from the dataset  
dataset=dataset.dropna()
```

✓ The Joint Plot

The `jointplot()` is used to display the mutual distribution of each column. You need to pass three parameters to `jointplot`. The first parameter is the column name for which you want to display the distribution of data on x-axis. The second parameter is the column name for which you want to display the distribution of data on y-axis. Finally, the third parameter is the name of the data frame.

```
# Let's plot a joint plot of age and fare columns to see if we can  
sns.jointplot(x='age', y='fare', data=dataset)
```

<seaborn.axisgrid.JointGrid at 0x7af401399430>

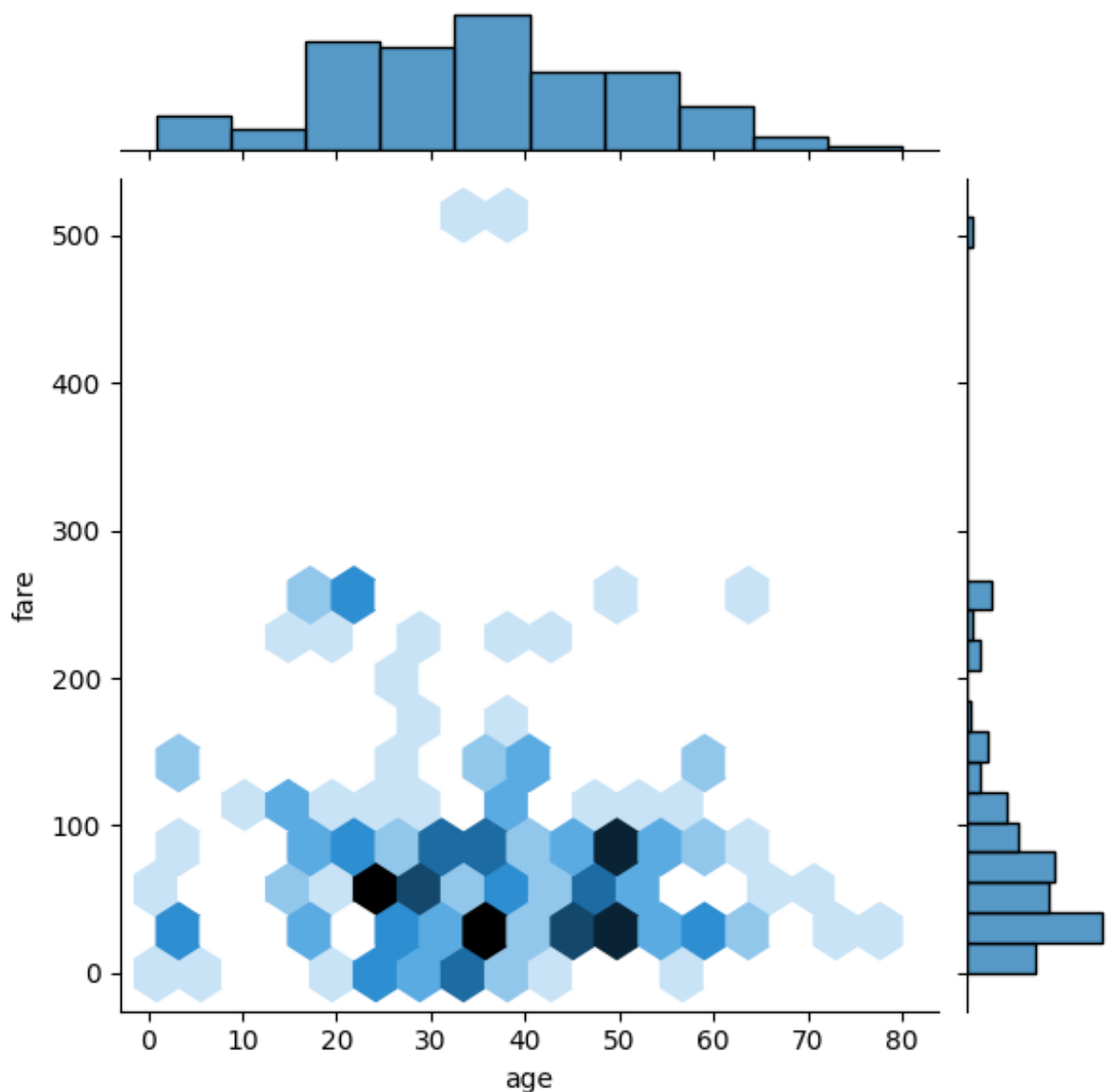


From the output, you can see that a joint plot has three parts. A distribution plot at the top for the column on the x-axis, a distribution plot on the right for the column on the y-axis and a scatter plot in between that shows the mutual distribution of data for both the columns. You can see that there is no correlation observed between prices and the fares.

You can change the type of the joint plot by passing a value for the kind parameter. For instance, if instead of scatter plot, you want to display the distribution of data in the form of a hexagonal plot, you can pass the value hex for the kind parameter.

```
sns.jointplot(x='age', y='fare', data=dataset, kind='hex')
```

```
<seaborn.axisgrid.JointGrid at 0x7af400ccbad0>
```



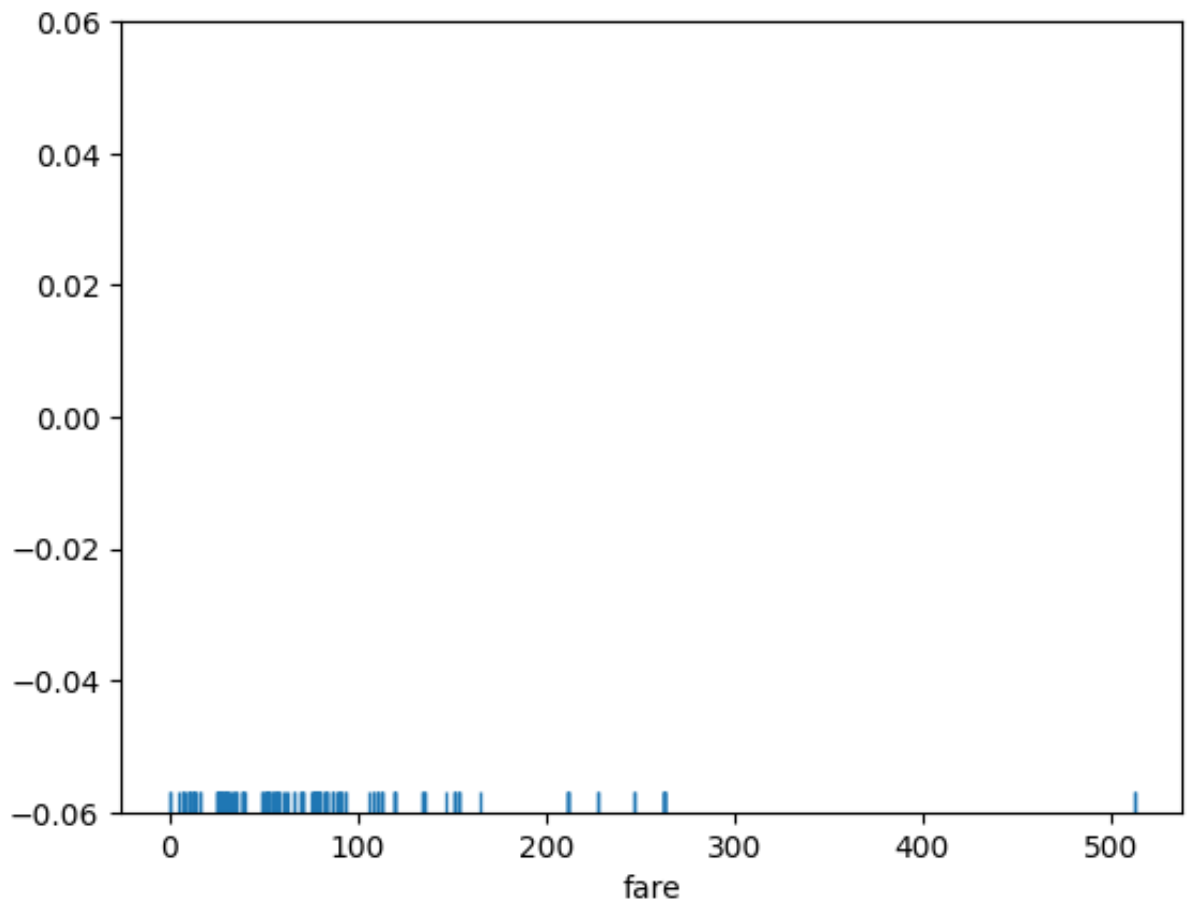
In the hexagonal plot, the hexagon with most number of points gets darker color. So if you look at the above plot, you can see that most of the passengers are between age 20 and 30 and most of them paid between 10-50 for the tickets.

✓ The Rug Plot

The rugplot() is used to draw small bars along x-axis for each point in the dataset.

```
# To plot a rug plot, you need to pass the name of the column. Let's  
sns.rugplot(dataset['fare'])
```

<Axes: xlabel='fare'>



you can see that as was the case with the `distplot()`, most of the instances for the fares have values between 0 and 100.

These are some of the most commonly used distribution plots offered by the Python's Seaborn Library.

Let's see some of categorical plots in the Seaborn library.

Categorical Plots

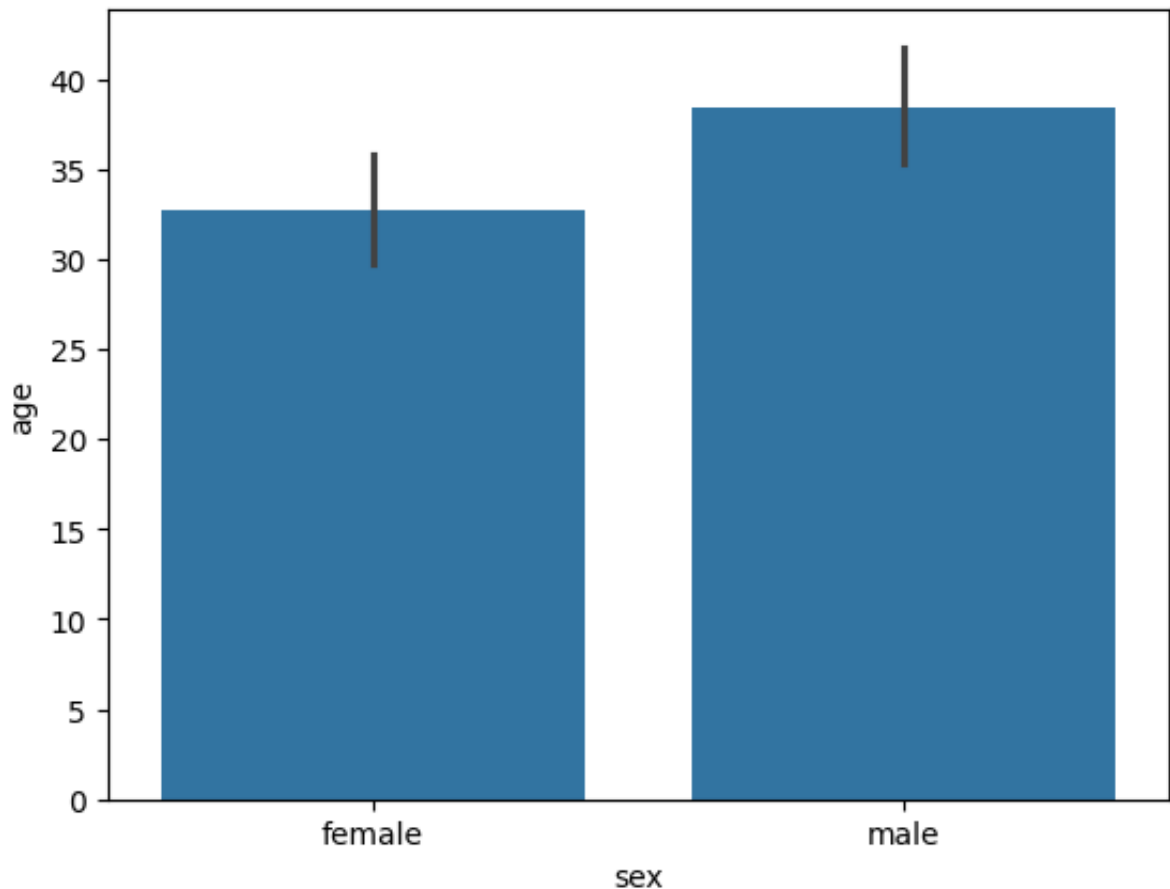
Categorical plots, as the name suggests are normally used to plot categorical data. The categorical plots plot the values in the categorical column against another categorical column or a numeric column. Let's see some of the most commonly used categorical data.

✓ The Bar Plot

The `barplot()` is used to display the mean value for each value in a categorical column, against a numeric column. The first parameter is the categorical column, the second parameter is the numeric column while the third parameter is the dataset.


```
# to know the mean value of the age of the male and female passeng  
sns.barplot(x='sex', y='age', data=dataset)
```

```
<Axes: xlabel='sex', ylabel='age'>
```



you can clearly see that the average age of male passengers is just less than 40 while the average age of female passengers is around 33.

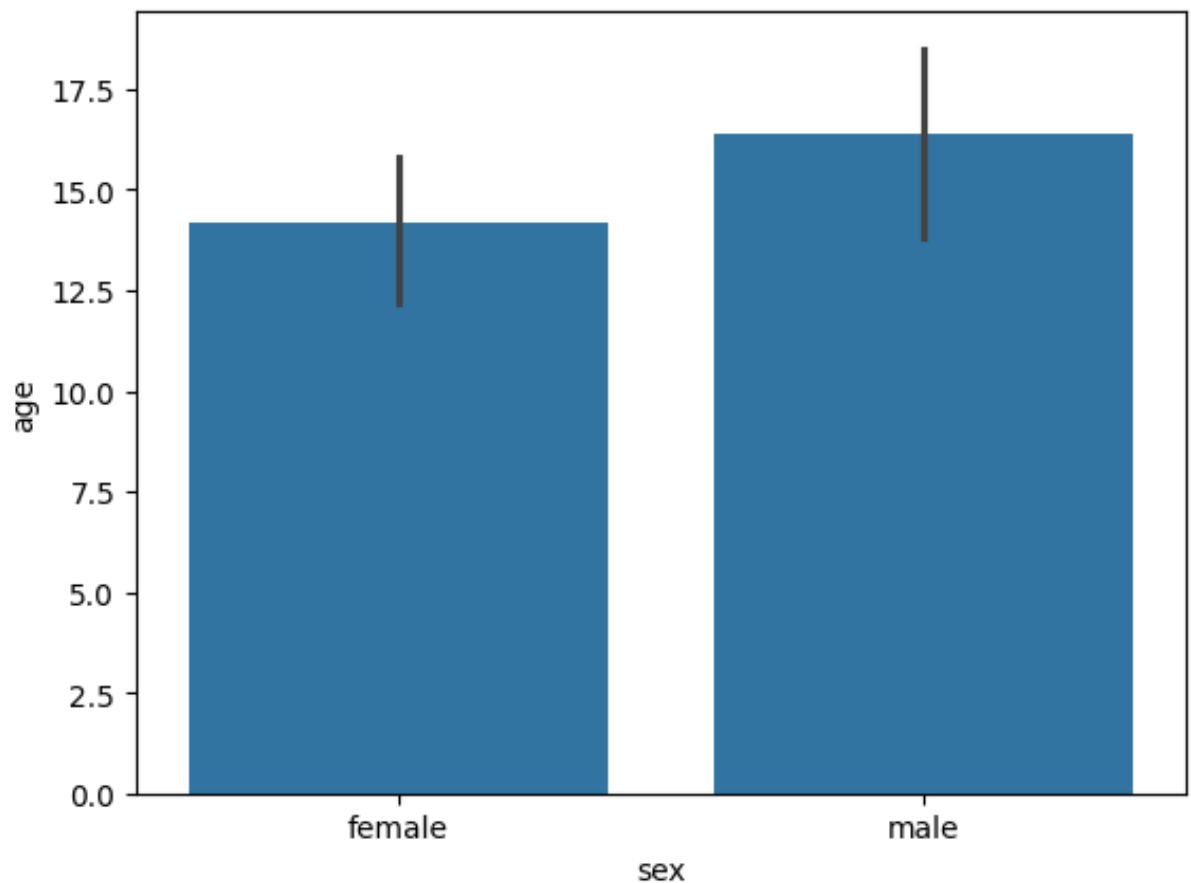
to finding the average, the bar plot can also be used to calculate other aggregate values for each category. To do so, you need to pass the aggregate function to the estimator.

```
# # To calculate the standard deviation for the age of each gender

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.barplot(x='sex', y='age', data=dataset, estimator=np.std)

# we use the std aggregate function from the numpy library to calcu

<Axes: xlabel='sex', ylabel='age'>
```

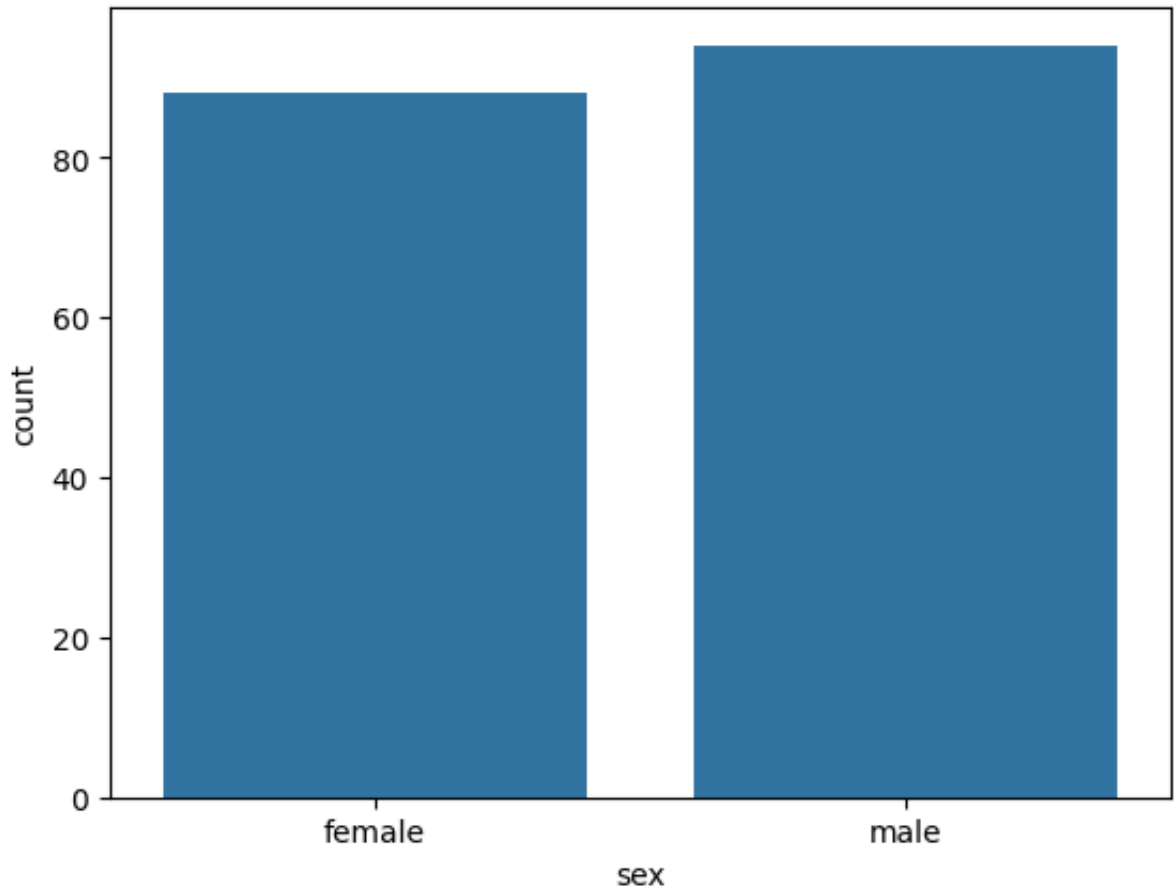


✓ The Count Plot

The count plot is similar to the bar plot, however it displays the count of the categories in a specific column.

```
# to count the number of males and women passenger we can do so use  
sns.countplot(x='sex', data=dataset)
```

```
<Axes: xlabel='sex', ylabel='count'>
```

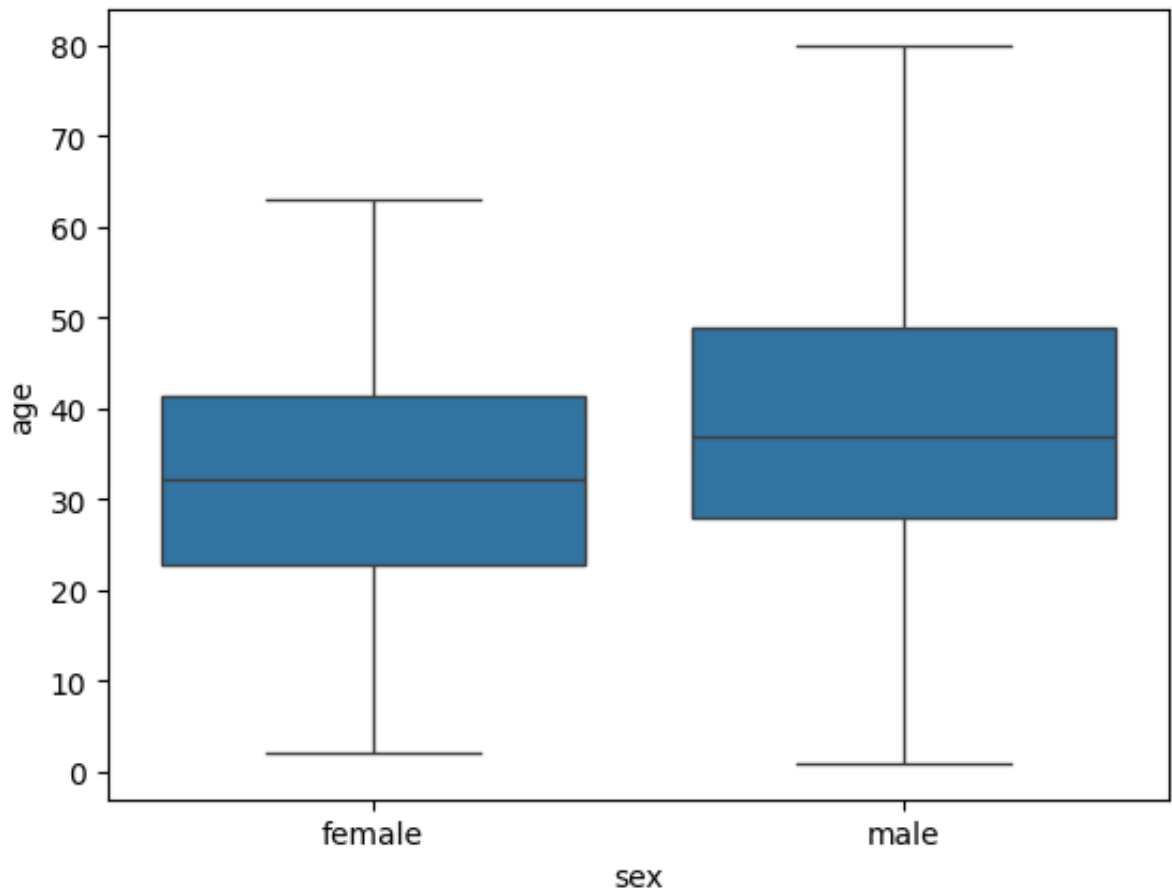


✓ Box Plot

The box plot is used to display the distribution of the categorical data in the form of quartiles. The center of the box shows the median value. The value from the lower whisker to the bottom of the box shows the first quartile. From the bottom of the box to the middle of the box lies the second quartile. From the middle of the box to the top of the box lies the third quartile and finally from the top of the box to the top whisker lies the last quartile.

```
# let's plot a box plot that displays the distribution for the age  
sns.boxplot(x='sex', y='age', data=dataset)
```

```
<Axes: xlabel='sex', ylabel='age'>
```

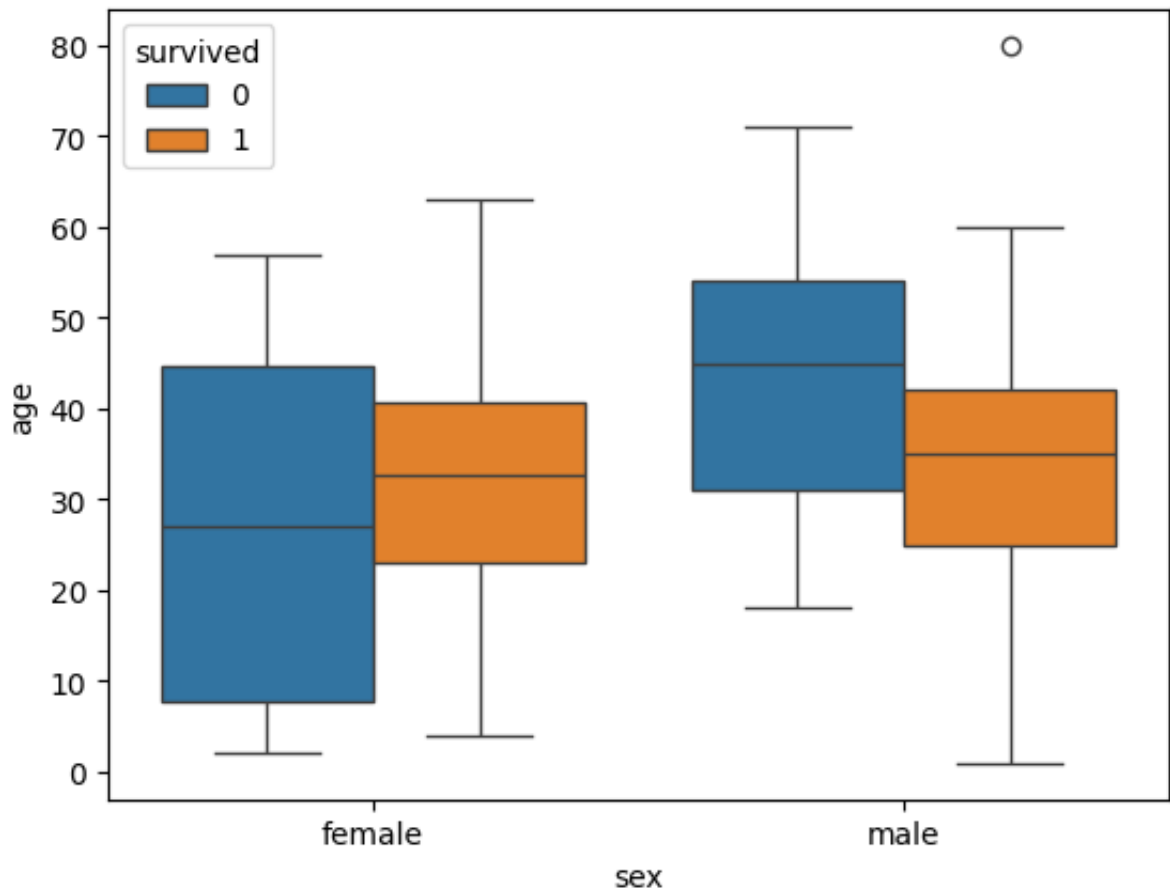


Let's try to understand the box plot for female. The first quartile starts at around 23 and ends at 42 which means that 25% of the passengers are aged between 23 and 42. The second quartile starts at around 23 and ends at around 32 which means that 25% of the passengers are aged between 23 and 32. Similarly, the third quartile starts and ends between 34 and 42, hence 25% passengers are aged within this range and finally the fourth or last quartile starts at 43 and ends around 65.

If there are any outliers or the passengers that do not belong to any of the quartiles, they are called outliers and are represented by dots on the box plot.

```
# if you want to see the box plots of forage of passengers of both  
sns.boxplot(x='sex', y='age', data=dataset, hue="survived")
```

<Axes: xlabel='sex', ylabel='age'>

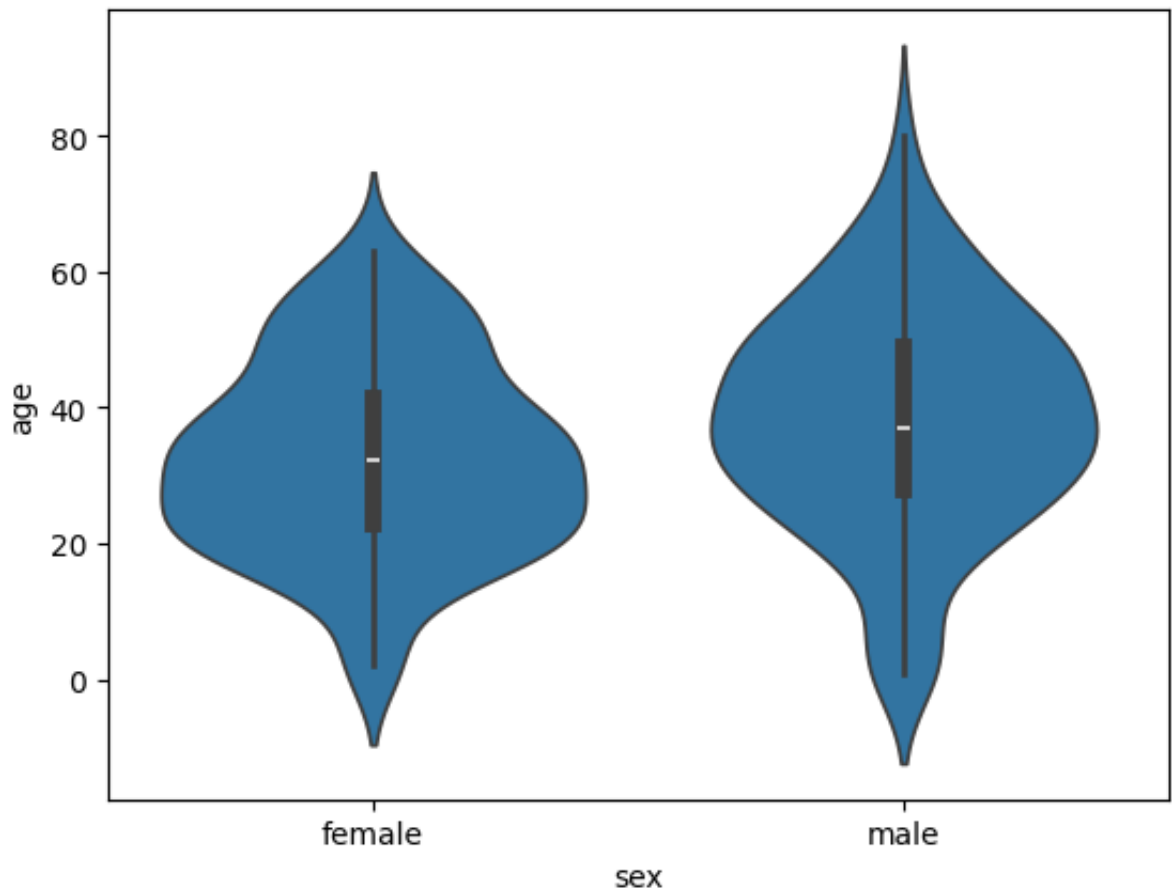


✓ The Violin Plot

The violin plot is similar to the box plot, however, the violin plot allows us to display all the components that actually correspond to the data point. The `violinplot()` function is used to plot the violin plot. Like the box plot, the first parameter is the categorical column, the second parameter is the numeric column while the third parameter is the dataset.

```
# Let's plot a violin plot that displays the distribution for the a  
sns.violinplot(x='sex', y='age', data=dataset)
```

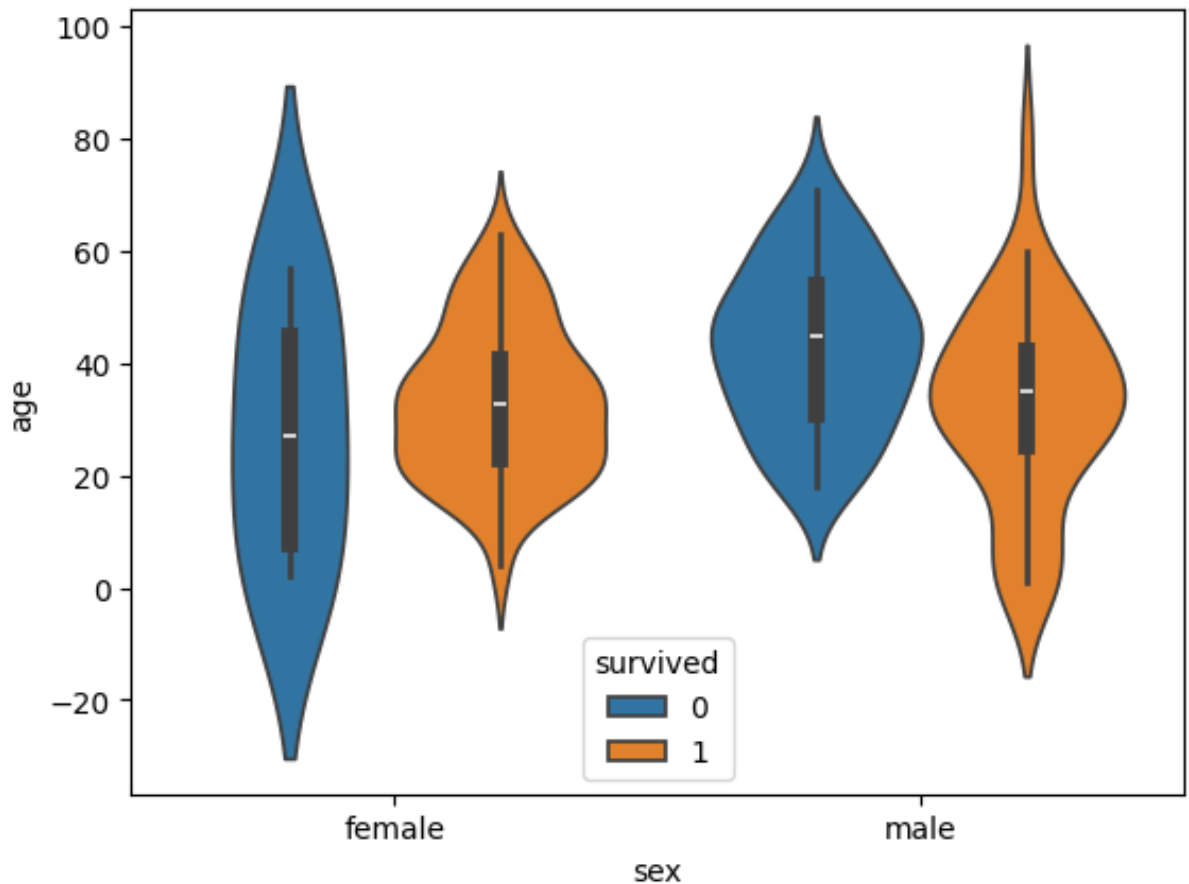
```
<Axes: xlabel='sex', ylabel='age'>
```



violin plots provide much more information about the data as compared to the box plot. Instead of plotting the quartile, the violin plot allows us to see all the components that actually correspond to the data. The area where the violin plot is thicker has a higher number of instances for the age.

```
# add another categorical variable to the violin plot using the hue  
sns.violinplot(x='sex', y='age', data=dataset, hue='survived')
```

```
<Axes: xlabel='sex', ylabel='age'>
```



Now you can see a lot of information on the violin plot. For instance, if you look at the bottom of the violin plot for the males who survived (left-orange), you can see that it is thicker than the bottom of the violin plot for the males who didn't survive (left-blue). This means that the number of young male passengers who survived is greater than the number of young male passengers who did not survive. The violin plots convey a lot of information, however, on the downside, it takes a bit of time and effort to understand the violin plots.

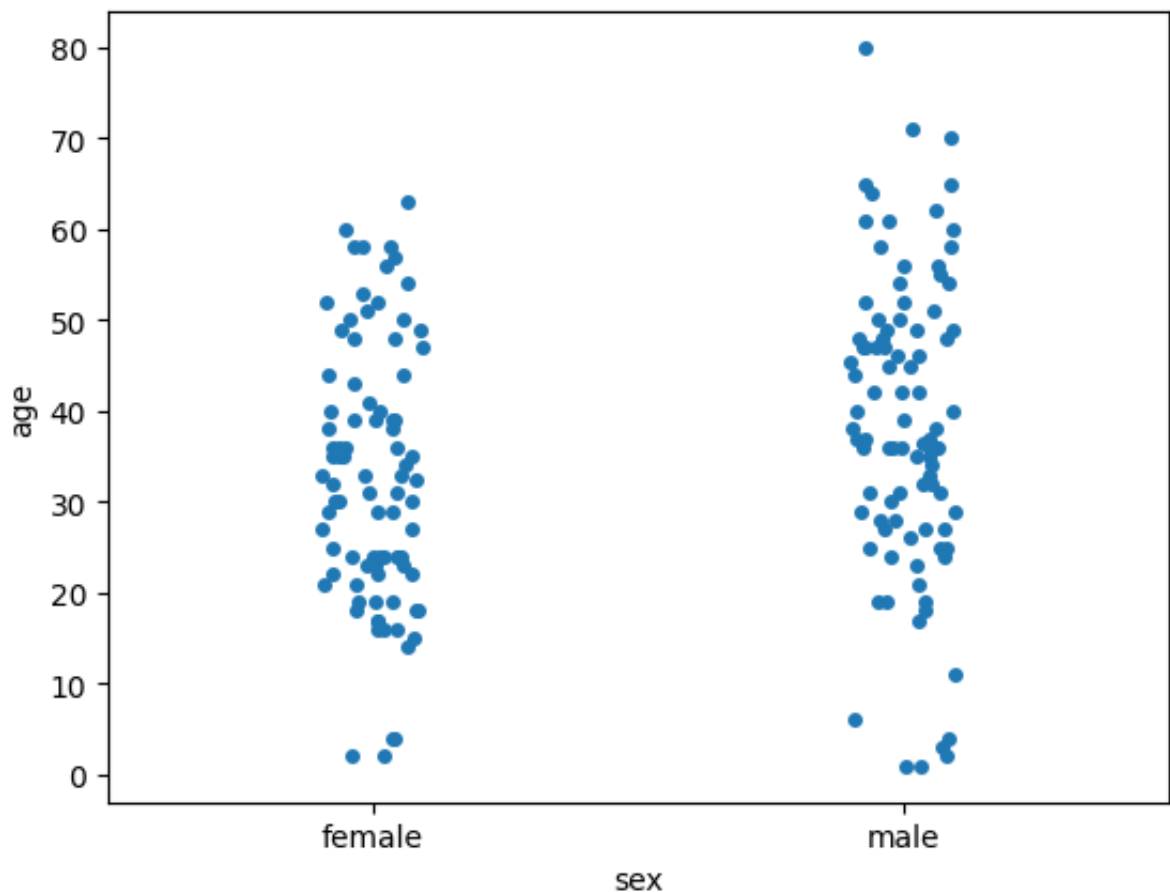
✓ The Strip Plot

The strip plot draws a scatter plot where one of the variables is categorical. We have seen scatter plots in the joint plot and the pair plot sections where we had two numeric variables. The strip plot is different in a way that one of the variables is categorical in this case, and for each category in the categorical variable, you will see scatter plot with respect to the numeric column.

The `stripplot()` function is used to plot the violin plot.

```
sns.stripplot(x='sex', y='age', data=dataset)
```

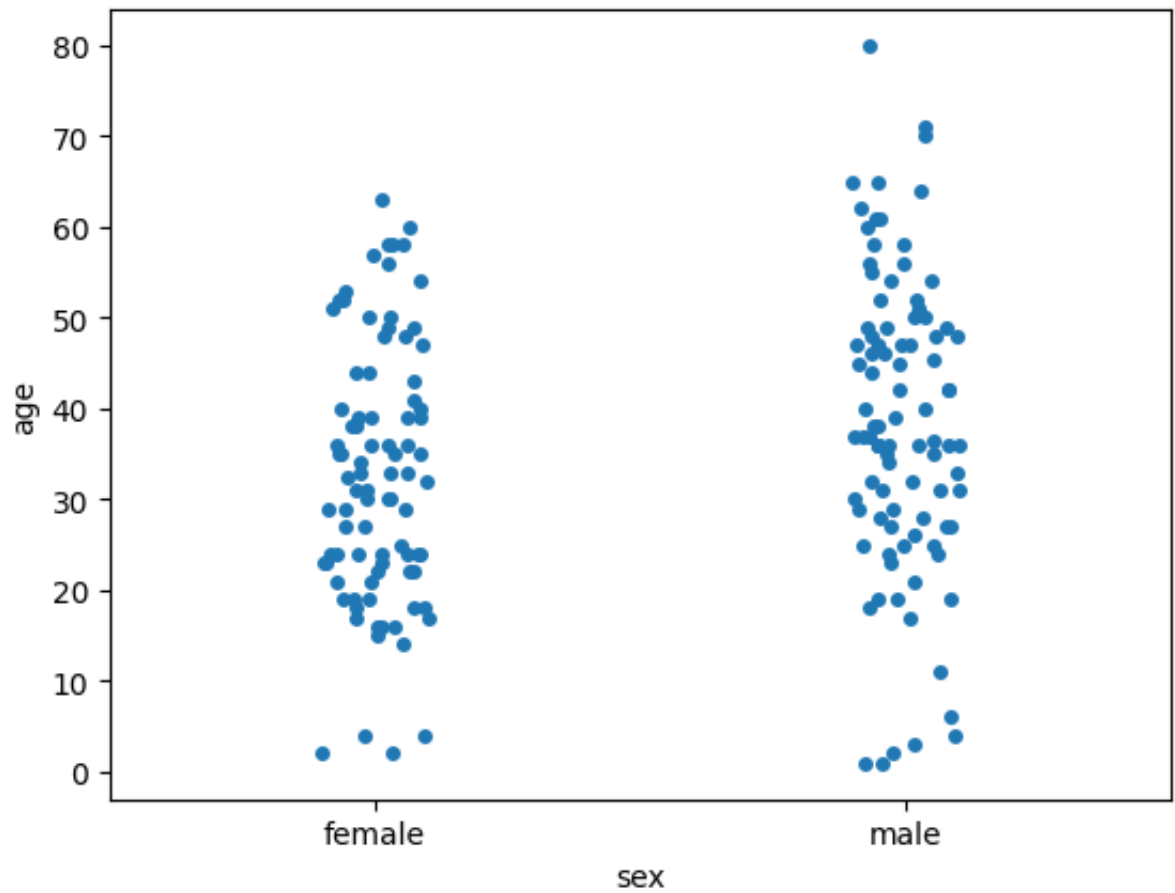
```
<Axes: xlabel='sex', ylabel='age'>
```



see the scattered plots of age for both males and females. The data points look like strips. It is difficult to comprehend the distribution of data in this form.

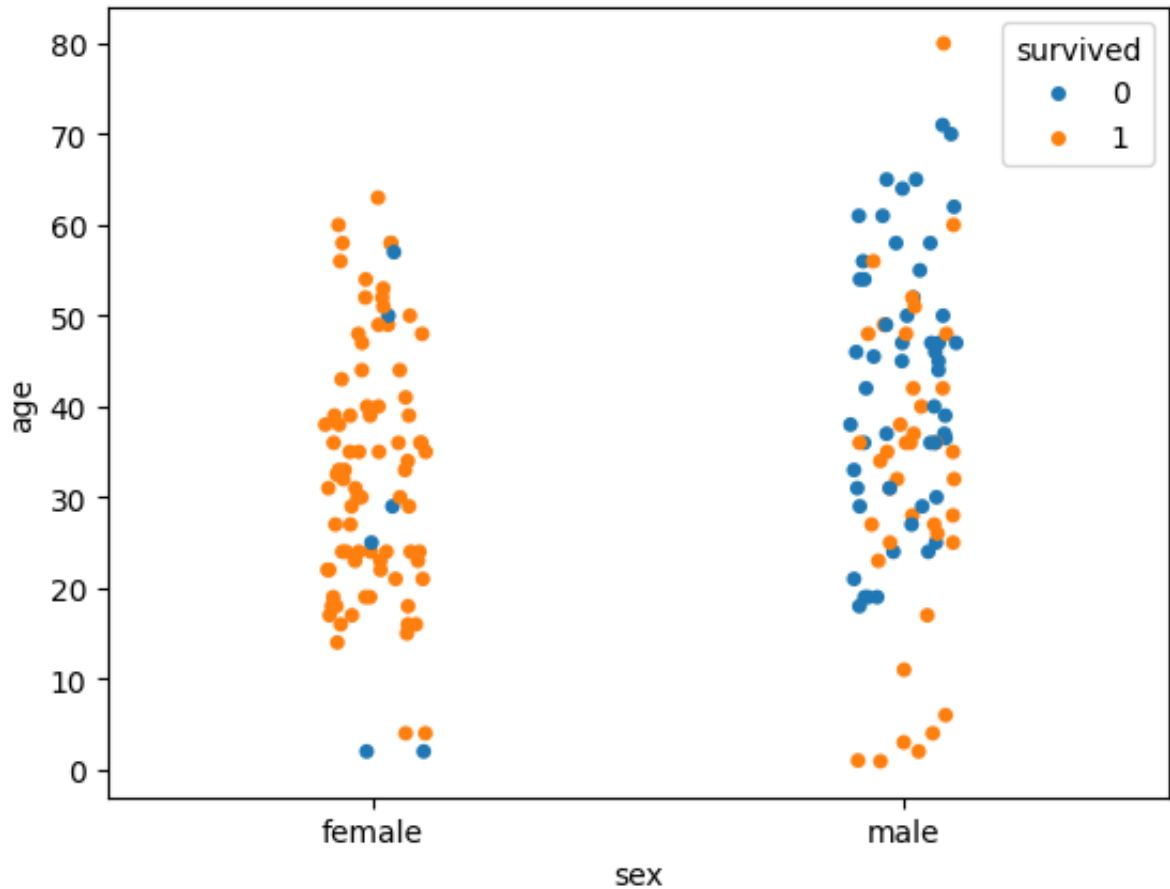

```
sns.stripplot(x='sex', y='age', data=dataset, jitter=True)  
#pass True for the jitter parameter which adds some random noise to
```

```
<Axes: xlabel='sex', ylabel='age'>
```



```
# Like violin and box plots, you can add an additional categorical  
sns.stripplot(x='sex', y='age', data=dataset, jitter=True, hue='sur
```

```
<Axes: xlabel='sex', ylabel='age'>
```



Seaborn is an advanced data visualization library built on top of Matplotlib library. In this above all code, we looked at how we can draw distributional and categorical plots using Seaborn library.

